

PROCESSAMENTO DE LINGUAGEM NATURAL

Projeto Final de Disciplina

Assistente de Estudos com RAG

Retrieval-Augmented Generation aplicado ao Contexto Acadêmico

1. Contextualização e Motivação

A explosão de documentos digitais — artigos científicos, manuais técnicos, legislações, relatórios corporativos e materiais didáticos — criou um desafio crescente: como encontrar a informação certa rapidamente e com confiança na sua veracidade?

Modelos de linguagem (LLMs) como o ChatGPT e o Claude são poderosos, mas têm uma limitação fundamental: eles respondem com base no que aprenderam durante o treinamento. Isso significa que podem **inventar informações** (fenômeno chamado de *alucinação*), não conhecem documentos privados e ficam desatualizados com o passar do tempo.

A técnica **RAG — Retrieval-Augmented Generation** (Geração Aumentada por Recuperação) resolve esses problemas combinando dois componentes:

RECUPERAÇÃO (Retrieval):

O sistema busca em uma base de documentos os trechos mais relevantes para a pergunta do usuário, usando busca semântica por embeddings.

GERAÇÃO (Generation):

O modelo de linguagem recebe esses trechos como contexto e gera uma resposta fundamentada nos documentos reais — reduzindo drasticamente as alucinações.

Este projeto propõe que você construa um **Assistente de Estudos com RAG** — um sistema que responde perguntas sobre um conjunto de documentos escolhido por você, demonstrando na prática todos os conceitos estudados ao longo do curso.

2. Objetivo Geral

Desenvolver um sistema completo de Assistente de Estudos baseado na arquitetura RAG, que seja capaz de:

1. **Indexar** uma base de documentos relevantes, dividindo-os em chunks com sobreposição
2. **Gerar** embeddings semânticos de cada chunk usando sentence-transformers
3. **Indexar** os vetores em um banco FAISS para busca eficiente por similaridade
4. **Recuperar** os chunks mais relevantes para qualquer pergunta do usuário
5. **Gerar** respostas fundamentadas nos documentos usando um LLM gratuito e local
6. **Manter** histórico da conversa para contexto contínuo entre perguntas
7. **Avaliar** a qualidade do sistema com métricas de Precision@k e Recall@k
8. **Comparar** RAG com LLM puro, demonstrando quando e por que o RAG é superior

3. Escolha do Tema e da Base de Documentos

Cada aluno deve escolher **um tema de interesse próprio** para construir a base de conhecimento do assistente. A escolha é livre, mas o tema deve ter documentos suficientes para ser útil.

3.1 Exemplos de Temas Aceitos

Área	Tema sugerido	Tipo de documento	Exemplo de pergunta
Direito	Código de Defesa do Consumidor	Legislação brasileira	"Qual o prazo para troca de produto com defeito?"
Saúde	Doenças crônicas e tratamentos	Artigos e bulas	"Quais os efeitos colaterais da metformina?"
TI	Documentação de frameworks	Docs técnicos (APIs)	"Como fazer autenticação JWT no Express?"
Finanças	Mercado financeiro e investimentos	Relatórios e artigos	"O que é diversificação de carteira?"
Literatura	Obras de domínio público	Livros (.txt)	"O que motivou a viagem de Ulisses?"
Meio ambiente	Legislação ambiental brasileira	Lei da natureza (PDFs)	"O que diz o Código Florestal sobre APP?"

3.2 Requisitos Mínimos da Base de Documentos

- **Mínimo de 5 documentos** (textos, PDFs ou arquivos .txt) sobre o tema escolhido
- **Mínimo de 2.000 palavras** no total da base de conhecimento
- **Conteúdo em Português** (preferencialmente) ou Inglês
- **Fontes identificadas e citáveis** — não use textos sem origem conhecida

Dica: onde encontrar documentos?

Legislação brasileira: planalto.gov.br, senado.leg.br
Artigos científicos: scholar.google.com (PDFs abertos), SciELO
Documentação técnica: docs.python.org, docs.djangoproject.com, etc.
Livros de domínio público: gutenberg.org, brasiliana.usp.br

4. Entregas Obrigatórias

4.1 Notebook Jupyter (.ipynb)

O notebook deve conter código funcional e comentado, organizado nas seguintes seções:

Seção	Conteúdo obrigatório	Peso
1. Base de documentos	Carregamento dos documentos com identificação de cada fonte	10%
2. Chunking	Função de divisão com <code>chunk_size</code> e <code>overlap</code> configuráveis, exibir estatísticas	10%
3. Embeddings + FAISS	Geração de embeddings, normalização e criação do índice	15%

4. Busca semântica	Função buscar() testada com pelo menos 5 perguntas, exibir scores	15%
5. Pipeline RAG	Prompt estruturado, geração com LLM gratuito, resultado completo	20%
6. Histórico de conversa	Memória entre turnos, demonstração com sequência de 3+ perguntas	10%
7. Comparativo RAG vs LLM	Pelo menos 3 perguntas mostrando diferença de qualidade	10%
8. Avaliação	Precision@1 e Precision@3 com gabarito de pelo menos 8 perguntas	10%

4.2 Relatório Escrito (PDF ou DOCX)

Documento de 4 a 8 páginas contendo:

1. Descrição da base de documentos: fontes, volume total, idioma
2. Resultados: métricas de avaliação obtidas com análise crítica
3. Comparativo RAG vs LLM puro: casos onde o RAG se mostrou superior
4. Limitações identificadas: o que o sistema não consegue responder bem?
5. Referências: fontes dos documentos usados na base de conhecimento

5. Estrutura Mínima do Notebook

O notebook deve ser organizado com as seguintes seções e marcadores markdown:

```
# Projeto 2 – Assistente de Estudos com RAG
## Tema: [SEU TEMA AQUI]
## Aluno(a): [NOME] | Turma: [X] | Data: [DD/MM/AAAA]

### Parte 1 – Carregamento dos Documentos
### Parte 2 – Chunking com Sobreposição
### Parte 3 – Embeddings e Índice FAISS
### Parte 4 – Motor de Busca Semântica
### Parte 5 – Pipeline RAG Completo
### Parte 6 – Histórico de Conversa
### Parte 7 – Comparativo: RAG vs LLM Puro
### Parte 8 – Avaliação de Qualidade (P@k)
### Parte 9 – Dashboard e Visualizações
```

Dúvidas frequentes

P: Posso usar o mesmo tema que outro aluno?

R: Sim, mas a base de documentos e o assistente devem ser diferentes.

P: Posso usar documentos em inglês?

R: Sim. Se usar, inclua na metodologia como isso afeta os embeddings e o LLM.

P: O LLM pode ser o ChatGPT via API?

R: Não. O projeto deve rodar localmente sem APIs pagas.

P: Posso usar uma biblioteca diferente de FAISS para o índice vetorial?

R: Sim, desde que seja gratuita e local (ex: ChromaDB). Justifique a escolha.

P: Meu top_k=3 não encontra boas respostas. O que faço?

R: Revise o chunking (talvez chunks grandes demais), tente outro modelo de embedding ou aumente o top_k.

Bom projeto! A melhor forma de aprender é construindo. Cada erro é um passo rumo ao entendimento real.